

For CentOS, type:

```
sudo yum install logrotate
```

Configuring logrotate

When logrotate runs, it reads its configuration files to decide where to find the log files that it needs to rotate, how often the files should be rotated and how many archived logs to keep. There are primarily two ways to write a logrotate script and configure it to run every day, every week, every month, and so on.

1. Configuration can be done in the default global configuration file `/etc/logrotate.conf`; or
2. By creating separate configuration files in the directory `/etc/logrotate.d/` for each service/application.

Personally, I think the latter option is a better way to write logrotate configurations, as each configuration is separate from the other. Some distributions use a variation and scripts that run logrotate daily can be found at any of the following paths:

- `/etc/cron.daily/logrotate`
- `/etc/cron.daily/logrotate.cron`
- `/etc/logrotate.d/`

One logrotate configuration (filename: Tomcat) file given below will be used to compress and take daily backups of all Tomcat `.log` files and `catalina.out` files and after rotation, the Tomcat service will get restarted. With this configuration it is clear that multiple log file backups can be taken in one go. Multiple log files should be delimited with space.

```
/home/tomcat/logs/*.log /home/tomcat/logs/catalina.out {
    missingok
    copytruncate
    daily
    compress
    rotate 10
    olddir /var/log/backup/tomcat/
    sharedscripts
    postrotate
        /home/tomcat/bin/catalins.sh restart > /
dev/null
    endscript
}
```

To check if the configuration is functioning properly, the command given below with the `--v` option can be used. Option `-v` means 'verbose' so that we can view the progress made by the logrotate utility.

```
logrotate -dv /etc/logrotate.d/tomcat
```



Figure 1: The logrotate utility

Logrotate options

<code>-d, --debug</code>	In debug mode, no changes will be made to the logs or to the logrotate state file.
<code>-f, --force</code>	This instructs logrotate to force the rotation, which is necessary as per logrotate: this is useful after adding new entries to a <code>config</code> file.
<code>-s, --state <statefile></code>	Tells logrotate to use an alternate state file. This is useful if logrotate is being run by a different user for various sets of log files. The default state file is <code>/var/lib/logrotate.status</code> .
<code>-m, --mail <command></code>	Tells logrotate which command to use when mailing logs. This command should accept two arguments: 1) the subject of the message, and 2) the recipient. The command must then read a message on standard input and mail it to the recipient. The default mail command is <code>/bin/mail -s</code> .
<code>-v, --verbose</code>	Turns on verbose mode.

The types of directives

Given below are some useful directives that can be included in the logrotate configuration file.

Missingok: Continues executing the next configuration in the file even if the log file is missing, instead of throwing an error.

nomissingok: Throws an error if the log file is missing.

compress: Compresses the log file in the `.tar.gz` format. The file can compress in another format using the `compresscmd` directive.

compresscmd: Specifies the command to use for log file compression.

compressext: Specifies the extension to use on the compressed log file. Only applicable if the compress option is enabled during configuration.

copy: Makes a copy of the log file but it does not make any modification in the original file. It is just like taking a snapshot of the log file.

copytruncate: Copies the original file content and then truncates it. This is useful when some processes are writing to the log file and can't be stopped.

dateext: Adds a date extension (default `YYYYMMDD`), to back up the log file. Also see `nodateext`.

dateformat format_string: Specifies the extension for `dateext`. Only `%Y %m %d` and `%s` specifiers are allowed.

Ifempty: Rotates the log file even if it is empty.

Also see `notifempty`.

olddir <directory>: Rotated log files get moved in the specified directory. Overrides `noolddir`.

sharedscripts: This says that `postscript` will run once for multiple configuration files having the same log directory. For example, the directory structure `/home/tomcat/logs/*.log` is the same for all log files placed in the `logs` folder, and in this case, `postscript` will run only once.